Java Programming Foundations

Comprehensive Lecture & Reference Notes for College Instructors & Students

Topic 1: Java Packages and Import Statements

In Java, packages are used to group related classes, interfaces, and sub-packages together. To utilize a class defined in another package, Java provides the `import` keyword. Understanding how imports work is essential for writing clean and modular code.

1.1 Types of Import Statements

There are two main types of explicit `import` statements in Java:

- **Specific Import:** Imports a single, explicitly named class from a package.

```
import javax.swing.JOptionPane;
```

- **Wildcard Import:** Imports all public classes within a specified package using the asterisk (`*`) wildcard character.

```
import javax.swing.*;
```

> **Performance Myth vs. Reality**
>
> **Is there a performance difference between specific and wildcard imports?**
> **No.** There is zero performance difference at compile time or runtime. Import statements merely tell the Java compiler where to locate class definitions. The compiler does not read or load class bytecode until the class is actually referenced and used in the code.

1.2 Implicit Imports and the `java.lang` Package

Java automatically imports a special core package into every Java program without requiring any `import` statement:

- The **`java.lang`** package is *implicitly imported* by default.
- Common classes such as `System`, `String`, `Math`, `Object`, and `Thread` reside in `java.lang`. This is why you can directly write `System.out.println()` without importing `java.lang.System`.

Topic 1 Assessment — Multiple Choice Questions

Q1. Which package is imported automatically into every Java program without explicit code? EASY

A) `java.util`

B) `java.lang`

C) `java.io`

D) `javax.swing`

Correct Answer: B) `java.lang` — It contains core classes required for basic language functionality.

Q2. What is the primary purpose of an `import` statement in Java? EASY

A) To load the binary executable files into memory at startup

B) To tell the compiler where to locate referenced classes in other packages

C) To copy the full implementation code of a library directly into your source file

D) To allocate heap memory for external objects

Correct Answer: B) To tell the compiler where to locate referenced classes in other packages.

Q3. Compare `import java.util.Scanner;` and `import java.util.*;` regarding performance. MEDIUM

A) Specific import is significantly faster at runtime.

B) Wildcard import slows down program execution speed.

C) There is no performance difference between specific and wildcard imports.

D) Wildcard import increases the compiled `.class` file size drastically.

Correct Answer: C) There is no performance difference between specific and wildcard imports.

Q4. Which Java statement displays a message "Hello World" in a graphical pop-up dialog box? MEDIUM

A) `System.out.println("Hello World");`

B) `JOptionPane.showMessageDialog(null, "Hello World");`

C) `Dialog.show("Hello World");`

D) `Window.print("Hello World");`

Correct Answer: B) `JOptionPane.showMessageDialog(null, "Hello World");` (from `javax.swing` package).

Q5. Suppose package `A` contains class `C1`, and sub-package `A.B` contains class `C2`. Does statement `import A.*;` allow direct access to `C2`? HARD

A) Yes, wildcard imports recursively import all nested sub-packages.

B) No, wildcard imports only import classes directly inside that specific package, not sub-packages.

C) Yes, provided `C2` is declared as public.

D) No, because sub-packages cannot contain public classes.

Correct Answer: B) No, wildcard imports do not recursively import classes inside sub-packages (you would need `import A.B.*;`).

Topic 2: Java Code Formatting, Indentation, and Block Styles

Writing clean, readable code is a essential skill for software developers. While Java compilers ignore white spaces and line breaks, human programmers rely on proper indentation and block formatting to comprehend program logic quickly.

2.1 Proper Indentation and Operator Spacing

Indentation visually indicates nesting and structural hierarchy. Best practices include:

- **Indent Levels:** Indent each nested block or statement by at least **2 to 4 spaces** relative to its enclosing structure.
- **Binary Operator Spacing:** Always place a single space before and after binary operators (e.g., `+`, `-`, `*`, `/`, `=`).

```
// Bad Formatting Style (Difficult to read)
System.out.println(3+4*4);

// Good Formatting Style (Clean spacing)
System.out.println(3 + 4 * 4);
```

2.2 Block Formatting Styles

A **block** in Java is a sequence of statements enclosed within curly braces (`{ }`). Two widely recognized block styling conventions exist in Java programming:

Style Name	Code Example	Characteristics

Next-Line Style	`public class Test`
	`{`
	`  public static void main(String[] args)`
	`  {`
	`  System.out.println("Hello");`
	`  }`
	`}`
		Opening brace aligns vertically directly below the declaration line. Easy to spot matching braces.

End-of-Line Style	`public class Test {`
	`  public static void main(String[] args) {`
	`  System.out.println("Hello");`
	`  }`
	`}`
		Opening brace is placed at the end of the declaration line. Saves vertical screen space; standard in Java API source code.

Golden Rule of Code Styling

Both Next-Line and End-of-Line styles are fully acceptable. However, **consistency is critical**. Choose one style and adhere to it throughout an entire project—never mix styles within the same file.

Topic 2 Assessment — Multiple Choice Questions

Q1. What defines a "block" of code in Java? EASY

A) A set of statements separated by commas

B) A group of statements surrounded by curly braces `{ }`

C) A single line of code ending with a semicolon

D) Statements enclosed within parentheses `( )`

Correct Answer: B) A group of statements surrounded by curly braces `{ }`.

Q2. Which block style places the opening curly brace { on the same line as the class or method declaration? EASY

A) Next-line style

B) End-of-line style

C) Free-form style

D) Inline-bracing style

Correct Answer: B) End-of-line style.

Q3. Why is consistent indentation important in Java programming? MEDIUM

A) The Java compiler will reject unindented code with a syntax error.

B) Indentation alters execution priority and runtime speed.

C) It visually illustrates structural relationships and enhances code readability for humans.

D) It reduces memory consumption of bytecode binaries.

Correct Answer: C) It visually illustrates structural relationships and enhances readability.

Q4. Which statement follows standard Java readability guidelines for binary operators? MEDIUM

A) `int result=a+b*c;`

B) `int result = a + b * c;`

C) `int result= a +b *c;`

D) `int result =a+b*c;`

Correct Answer: B) `int result = a + b * c;` (spaces surrounding binary operators).

Q5. Which block style is used by default throughout the official Java API standard library source code? HARD

A) Next-line style

B) End-of-line style

C) Tab-aligned style

D) K&R C-style variant without braces for single lines

Correct Answer: B) End-of-line style is standard in the Java API code base.

Topic 3: Categorization of Programming Errors

Errors occur naturally during software development. In Java programming, errors fall into three main classifications: **Syntax Errors**, **Runtime Errors**, and **Logic Errors**.

3.1 Syntax Errors (Compile-Time Errors)

Syntax errors result from violations of Java language grammar, keyword spelling, or punctuation rules. They are detected by the compiler (`javac`) before program execution.

- **Common Causes:** Missing semicolons, unmatched curly braces, unclosed string literals, mistyped keywords (e.g., `viod` instead of `void`).
- **Debugging Tip:** Fix compiler errors starting from the *top line downwards*. A single syntax error early in the code often triggers a cascade of false error messages below it.

```
// Syntax Error Example
public class ShowSyntaxErrors {
    public static main(String[] args) { // Error: 'void' keyword missing
        System.out.println("Welcome to Java); // Error: Unclosed string literal
    }
}
```

3.2 Runtime Errors

Runtime errors occur while the program is actively executing. The code compiles cleanly, but an illegal operation causes the program to terminate abruptly (throw an exception).

- **Common Causes:** Division by integer zero (`ArithmeticException`), accessing array indices out of bounds, or unexpected user input type mismatches.

```
// Runtime Error Example
public class ShowRuntimeErrors {
    public static void main(String[] args) {
        System.out.println(1 / 0); // Causes ArithmeticException: / by zero
    }
}
```

3.3 Summary Comparison of Error Types

Error Type	When Detected	Program Execution	Primary Cause
Syntax Error	Compile time (by `javac`)	Does not start (fails to build)	Violating Java language grammar / syntax rules
Runtime Error	Runtime (during execution)	Terminates abruptly / crashes	Illegal operation (e.g. division by zero)
Logic Error	Testing / Output verification	Runs to completion with wrong results	Flawed algorithm or logic implementation

Topic 3 Assessment — Multiple Choice Questions

Q1. Which type of error is detected by the Java compiler prior to running the application? EASY

A) Runtime error
B) Logic error
C) Syntax error
D) Memory leak

Correct Answer: C) Syntax error (also referred to as compile error).

Q2. Executing `System.out.println(10 / 0);` in Java produces what type of error? EASY

A) Syntax Error
B) Runtime Error (`ArithmeticException`)
C) Logic Error
D) Compilation Warning

Correct Answer: B) Runtime Error — Division by integer zero terminates the execution abnormally.

Q3. What is the recommended strategy when dealing with multiple compiler syntax error messages? MEDIUM

A) Fix errors from the bottom error message upwards.
B) Fix the very first reported error from the top down and recompile.
C) Ignore syntax errors and run the class file anyway.
D) Delete the class file and restart.

Correct Answer: B) Fix errors starting from the top line downward, as early errors cause cascading false errors.

Q4. A program compiles cleanly and runs without crashing, but calculates incorrect tax totals. What error is present? MEDIUM

A) Syntax Error
B) Runtime Error
C) Logic Error
D) Hardware Error

Correct Answer: C) Logic Error — The program executes but produces incorrect logic outputs.

Q5. Which of the following code snippets will generate a Syntax Error at compile time? HARD

A) `double result = 1.0 / 0.0;`
B) `String s = "Hello World;`
C) `int x = Integer.parseInt("abc");`
D) `int[] arr = new int[5]; System.out.println(arr[5]);`

Correct Answer: B) `String s = "Hello World;` due to an unclosed string literal (Option A results in `Infinity`; C and D are runtime exceptions).